cw2 average

Matching & Image Geometry

Hao Zhao
Imperial college & MSc CSP
CID: 01220907
hz8315@ic.ac.uk

Robin Dupont
Imperial college & MSc CSP
CID: 01277640
rqd16@ic.ac.uk

***Abstract*—This report illustrates the harris detection and feature matching at first, and then reveals several methods to detect image transformation. Finally, the homography accuracy(HA) and projecting for images taken by ourselves will be illustrated, and epipolar lines of matched image will be shown.**

I. Automatic

A. Corner detection

1) Harris: Both edges and corners are considered as features in computer vision, and corners are commonly used as interest points for image matching, because corner has significant change in all directions and rotate invariant. In this part, we compared two ways to detect corner, one way is from Harris, and the other approach is to detect edges at first and then detect the intersections of edges.

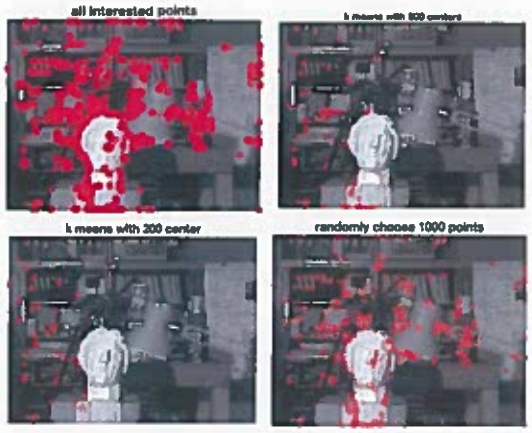


Fig. 1. Harris detection

After implementing Harris detection algorithm, the threshold on result was set as 10^6, as different corners have various values and higher threshold may result in losing some corners. However, one can see a problem on first image in Fig. 1, the total number of results is larger(around 9000), because many closely located points cluster together around each theoretical corner. In other words, total 9000 points may only have 1000 unique corners. So k-means was tried to see if centers of every cluster could be found to represent one corner. But the performance seems not good that only [illegible]s on left side left, which might due to corners at lower image have similar R values so they were considered as members in upper clusters. Furthermore, if only choose 1000 points out of total randomly, the result is better which can be seen in last image, it contains most of unique corners and no longer cluster. Therefore, if the number of corners not need to be that high, one could only use few hundreds of randomly selected points to avoid clustering problem.

2) Intersections on edge: Fig.2 shows the corners detected by intersections of edges, it has the same cluster problem as Harris. And when 1000 points were chosen randomly, the result is worse than Harris, because all of points are spread out, which is like edge detection rather than corners. And if only choose first 200 largest points, the result is better as most of points represent corners in image. From this point, Harris is more accurate and is able to detect more corners than intersections of edge.



Fig. 2. Intersections of edges

B. Feature representation

If image patch used to describe each corner, intensity histogram or randomly chosen M intensity values could be used to achieve dimension reduction. Fig. 3 reveals two sizes

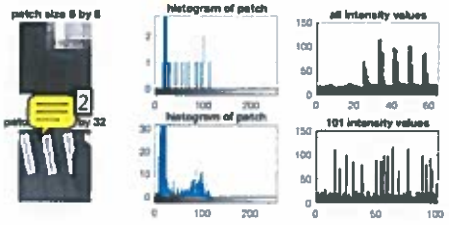


Fig. 3. Corner feature representation

of patch representation for corner of book. When the patch size is small(8×8), one can see a clear unique corner, but when patch is larger(32×32), there could be many corners in each patch, which might result in ambiguous description for closely located interested points. Moreover, theoretically, histogram should be better than randomly chosen intensity values in terms of description, as it provides a clear intensity

Summary of Comments on mlcv_cw2_average.pdf

Page: 1

Number: 1 Author: . Subject: Sticky Note Date: 22/05/2017 11:40:50

General: Language could be clearer, and do not use contractions when doing a report.
1) Good examples, but playing with the threshold (or using a threshold for the distance in NN) would be better than randomly selected points.
2) Good testing on known images. HA is mean euclidean distance. Epipolar line part a bit confusing.
3) Show matrix. SSD confusing, use HA (which is a mean of Euclidean distances).
4) No focal length change, put original images in the document, not in the appendix. Disparity map without range of values.

Number: 2 Author: . Subject: Sticky Note Date: 22/05/2017 11:18:54

Good figure showing example.

Number: 3 Author: . Subject: Sticky Note Date: 22/05/2017 11:18:01

It is good to test different approaches (like kmeans), but instead of selecting random points maybe playing with the threshold would be better.

distribution. And also there is no clear information on random intensity values images, as can be seen in Fig. 3.

C. Matching Feature

In order to have better visual effect on matching points, only 200 corners were chosen.



Fig. 4. Feature matching

Nearest neighbor has been used to implement feature matching, and Fig.4 illustrates matching results for three kinds of features. All three features seems good for matching, but histogram of 32×32 patch needs higher distance threshold and it has less matched pairs compared with another two. In other words, the accuracy of it is relatively lower. Moreover, one can see that for third feature(all 64 intensity values in 8×8 patch) gives more pairs but has more errors(slash), which might due to the patch size is small. Therefore, randomly chosen 101 values in 32×32 patch gives best result, able to provide enough matched pairs and less errors.

II. Transformation Estimation

A. Similarity transformation

For simple transformation, like horizontal-only translation in **tsukuba** image set, one could calculate the difference between matched points between two images directly, and the most repeated difference along horizontal direction is the expected result. By using the code attached in appendix, for two consecutive **tsukuba** images(Fig. 4), obtained coefficient t_x in translation matrix is 5 pixels, and t_y is 0. Moreover, after using function for rotation and scaling matrix in appendix, the coefficients of α and s for two consecutive images (Fig. 6 in appendix)in **graf** image set are calculated as 8.1127^o and 1 respectively, which means the image rotates 8.1127^o anti-clockwise and is not scal

B. Fundamental homogeneous matrix

For more complex transformation, like affine and projective transformation, general homogeneous matrix can be used to find out how image transformed. The code and how it work were attached in the appendix. Moreover, it gives the same results for translation matrix and rotation matrix as in last subsection. And also the code for fundamental matrix was attached, it was derived by applying single value decomposition on matrix $\mathbf{A}(\mathbf{AF} = 0)$.

C. Projecting and HA

The method for projecting image B to A for rotation transformation was done in this section and the code for it is in appendix. As Fig. 5 shown, one horizontal **graf** image was rotated 5° clock-wise. And because the rotated image is in the same size, so some information will be lost due to out of range. Moreover, the center of original image does not move, in other words, this projecting simulates rotating camera but keeping same camera location and focus.

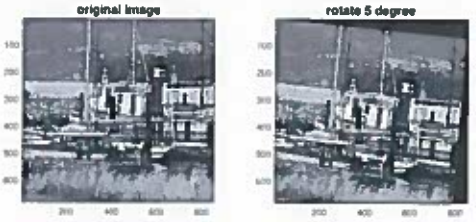


Fig. 5. Rotation projecting

For HA, the approach is similar to **RANSAC**, a few best matched points were chosen to find out homography at first, and then how many points out of total fit that homography is tested. In this part, translation accuracy in two consecutive **tsukuba** images was explored, 2 visually best matched points were chosen and average euclidean distance was detected as 5 pixels. Furthermore, as every pixel is too small and depths of objects between camera are different, $\pm$ 3 pixels error is acceptable in this case. Finally, the number of correct results among the rest of 109 pairs are shown in table below:

dis	2	3	4	5	6	7	8
num	0	0	2	46	14	0	14
HA_{total}	70%						

TABLE I
HA FOR TWO CONSECUTIVE TSUKUBA IMAGES

D. Epipolar line

For translation transformation, matched point must moves in the line which covers that point and with the gradient $\frac{t_y}{t_x}$. In this section, t_x and t_y were chosen as 3 and 5 respectively, and the epipolar line for three corner points are plot in Fig. 6.

Page: 2

Number: 1 Author: . Subject: Sticky Note Date: 22/05/2017 11:25:13
Ok

Number: 2 Author: . Subject: Sticky Note Date: 22/05/2017 11:19:54
32x32 is better, but instead of random points, using thresholds would be good. A threshold of distance in NN for example.

Number: 3 Author: . Subject: Sticky Note Date: 22/05/2017 11:26:48
HA is mean Euclidean distance, not error.

Number: 4 Author: . Subject: Sticky Note Date: 22/05/2017 11:28:31
What do you mean by chosen? An image from tsukuba was translated those pixels?

As the figure shown, all three green circle points must move along the corresponding three red lines.

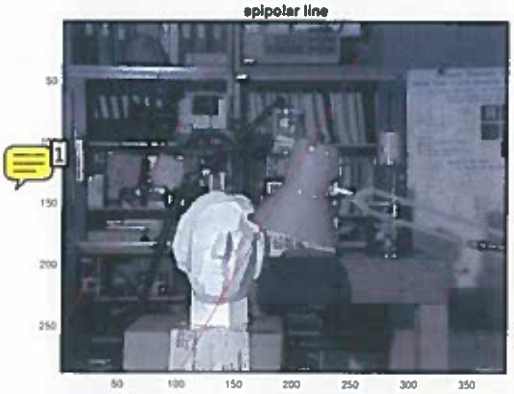


Fig. 6. Epipolar lines for one tsukuba image

III. Homography

A. Corner detection

Figure 15 in appendix shows 100 randomly selected corners detected by harris(code in appendix), for an HG image and resized version (reduced by a factor of 2). From the image, most of the interesting points appears in both images are in same locations, but the descriptors bound to small details, such as the text on the left of the pictures, doesn't appear on the resized image. The lost in resolution prevent the descriptors from finding these small details on the resized image. Therefore, compared with perfect manual method, harris detection on lower resolution image does not have effect on points location but affects the number of detected corners.

After corner detection, 101 randomly chosen intensity values in $32 \times$ ch were used as descriptor for NN matching. Table 2 illustrates SSD for NN matching and manually matching, for both of them original pair images have higher SSD which are around twice times of resize ones, because the resizing scale is half. And original images are matched roughly as good as resized ones, as can be seen in Fig. 17 and Fig. 18. Furthermore, SSD by manually matching is larger which m due to only 10 pairs of points were used.

NN matching		Manual matching	
original	resized	original	resized
190	98	206	100

TABLE II
SSD ON MATCHING HG IMAGES

B. Projecting

Fig. 7 and Fig. 8 reveal projected points by homography estimated by manual chosen points and automatically detected matched points. Both two give good projecting result which means both two homography are estimated correctly. However, as only 4 pairs were chosen among all automatically detected pairs, if wrong match included, projecting performance is much worse. When transformation is more complex, homography can also be estimated as can be seen in Fig. 16. One can see that most of points are matched correctly, except some points on keyboard which might due to transformation on keyboard(projective) is more complex than it on notebook(affine). In other words, one homography is not enough to describe all transformation on Fig. 16. With the homography matrix H it's possible to extract informations about the transformation. For example by applying $\sin^{-1}$ to $\mathbf{H}(1,2)$ the rotation angle can be extracted. If the second image is scalled with factor s, $\mathbf{H}(1,2)$ should be devided by s. Both the automated and manual methods allows to find an angle of ~ 9 degrees. The coefficients $\mathbf{H}(1,3)$ and $\mathbf{H}(2,3)$, should represent the t_x and t_y translation. However values given by both automatic and manual modes are quite high to be considered as correct (between 1000 and 100 pixels according to the cases).



Fig. 7. Homography detected by matched points obtained manually



Fig. 8. Homography detected by matched points obtained automatically

C. HA error

One pair of original HG images were used in this section, and 125 pairs of interest points were matched. Fig. 9 shows SD for each pair and mean SSD from 1 pair to total 125 pairs. From SD plot, one can see that most of SD fluctuate around 180 which is close to 206 by manually matching in table 2, but some pairs provide SD even larger than 1000 which can be regarded as outliers(wrong match). Considering SSD plot, original SSD plot is quite large at the beginning and ends around 280, and after removing all outliers(SD larger than 1000), the SSD plot is more flat and ends around 200 which

Page: 3

Number: 1 Author: . Subject: Sticky Note Date: 22/05/2017 11:29:29
If translation is vertical and horizonal, ok

Number: 2 Author: . Subject: Sticky Note Date: 22/05/2017 11:29:37
Do not use contractions in a formal report

Number: 3 Author: . Subject: Sticky Note Date: 22/05/2017 11:32:42
Show matrix

Number: 4 Author: . Subject: Sticky Note Date: 22/05/2017 11:31:32
Not good descriptor, use histogram instead for an easy descriptor or SURF/SIFT/FREAK for better descriptors

Number: 5 Author: . Subject: Sticky Note Date: 22/05/2017 11:34:04
Use HA. Why SSD? Better to do the mean

is close to 206 in table 2. From this point, simple average SSD over total pairs can not discard wrong match and it is better to use SSD ratio instead of one SSD only.

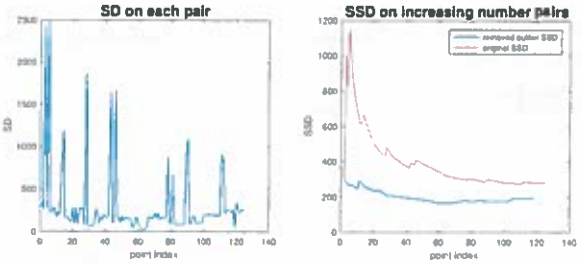


Fig. 9. SD and SSD of matched points in original images

IV. Stereo Vision

A. Estimating fundamental matrix and displaying epipolar lines

After using code in appendix, the fundamental matrix for FD images was estimated by automatically matched points, and then epipolar lines for $\mathbf{x}$ from first image and $\mathbf{x}'$ from second image were simulated by $\mathbf{Fx}$ and $\mathbf{F}^T\mathbf{x}'$. Fig. 10 shows the epipolar lines and epipoles, from the image, all of lines on two images have same gradient as epipolar planes are parallel with each other. However, lines are not horizontal perfectly, because the camera on phone was shaken during taking photo.

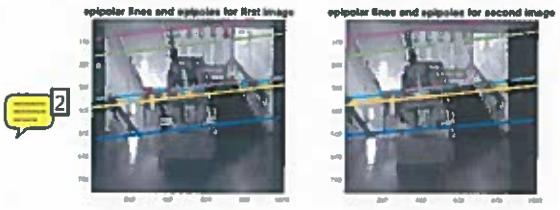


Fig. 10. Epipolar lines and epipoles for FD images

B. Disparity map

Disparity is the pixel difference between corresponding point horizontally, for the object closer to camera, disparity is larger than that compared with the object more far away. However, one important constraint is each object should has different color between other objects, otherwise, obtained matched point is wrong and will result in noise. Therefore, one of the solution is to apply k-mean clustering to to image color quantization before matching. The most important point is that the images should be rectified, it is to say that interest points should be on the same rows for each images. Figure 11 shows a disparity map created with images visible on Figure 20. The brighter the pixel is, the closer from the camera it is. Disparity map can be improved by image processing techniques like closing. the result is shown on Figure 22.

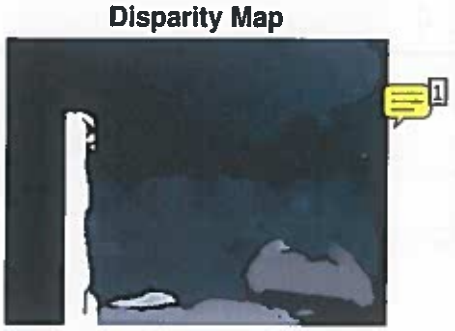


Fig. 11. Disparity map

C. 3D Depth map

Depth describes distance between object and camera which is inversely proportional to disparity. Fig. 12 shows the depth map generated from the disparity map. A Textured version of this depth map is visible in appendix, Fig 19.

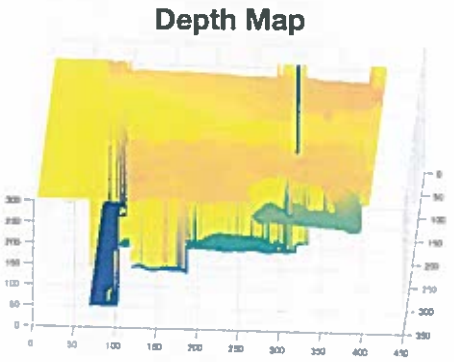


Fig. 12. Depth map

D. Focal length and noise

A noisy Depth map is shown on Figure 21. If the noise level is lower that a certain depth resolution, it is still possible to see some details, such as the sreen and the chair in this example. However, the tiny details are sunk into the noise and therefore invisible.

V. Conclusion

This report reveals the image matching at first, harris detection was used to detect corner, then three different kinds of feature descriptions were compared for NN matching, and the approach that randomly chosen 101 intensity values gave best match performance. Moreover, SIFT could also be implemented to compare in future. After that, homography matrix was estimated for different kinds of rotation and fundamental matrix was used to plot epipolar lines. Finally, the constraints of disparity map and depth map were discussed.

Page: 4

Number: 1 Author: . Subject: Sticky Note Date: 22/05/2017 11:38:48
Hard to see disparity. Put range of values. Show original images, do not put them in the appendix

Number: 2 Author: . Subject: Sticky Note Date: 22/05/2017 11:36:27
Ok, makes sense if translation if horizontal and vertical (show FD matrix too)

Number: 3 Author: . Subject: Sticky Note Date: 22/05/2017 11:40:35
Focal length change?

ACKNOWLEDGMENT

The authors would like to express deepest appreciation to Mr. Guillermo Garcia-Hernando and Dr. Vassileios Balntas who provided the suggestion, helped us to coordinate coursework and report.

APPENDIX

AUTOMATIC

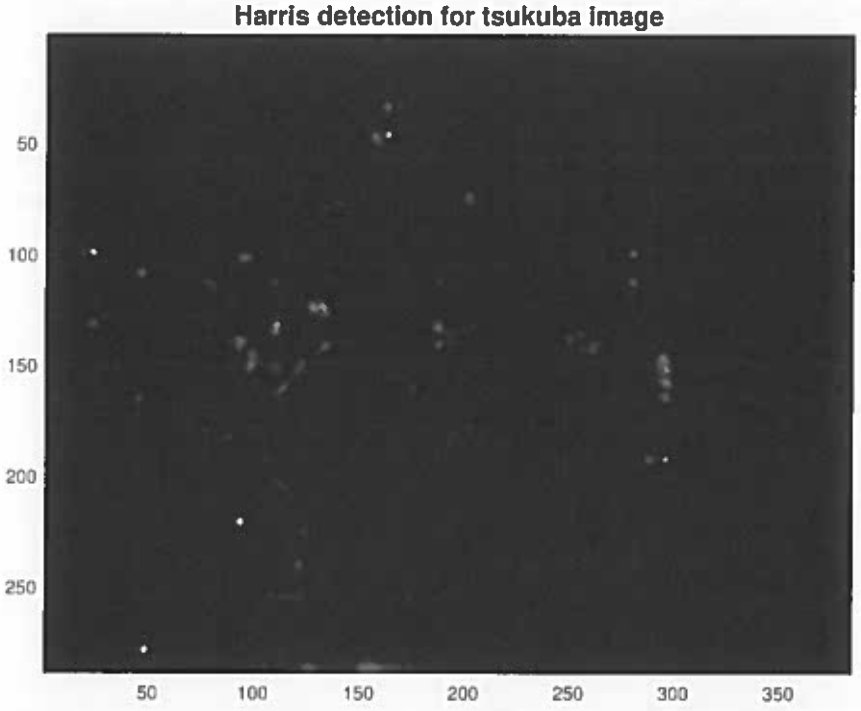


Fig. 13. Harris detection result for one tsukuba image



Fig. 14. Feature matching for rotated image

It is more difficult to match features for rotated images, as can be seen in figure above, only two corners match successfully.

Here is the code for harris detection:

```
window=[0.03 0.03 0.03 0.03 0.03 0.03 0.03;
        0.03 0.105 0.105 0.105 0.105 0.105 0.03;
        0.03 0.105 0.222 0.222 0.222 0.105 0.03;
        0.03 0.105 0.222 0.286 0.222 0.105 0.03;
        0.03 0.105 0.222 0.222 0.222 0.105 0.03;
        0.03 0.105 0.105 0.105 0.105 0.105 0.03;
        0.03 0.03 0.03 0.03 0.03 0.03 0.03;];
% window coefficients


%% harris detection
I=imread('2.jpg'); % load image here
I=rgb2gray(I);
I=double(I);

% figure,imagesc(I),colormap gray;
h_detector=[1 0 -1];
I_h = abs(imfilter(I,h_detector));
v_detector=[1 0 -1]';
I_v = abs(imfilter(I,v_detector));

Ih=I_h.^2;
Iv=I_v.^2;
Ihv=I_h.*I_v;

window_h=imfilter(Ih,window);
window_v=imfilter(Iv,window);
window_hv=imfilter(Ihv,window);

for i=1:(size(I,1))

    for j=1:(size(I,2))


        M=[window_h(i,j),window_hv(i,j);
           window_hv(i,j),window_v(i,j)];

        R(i,j)=det(M)-0.05*trace(M);

    end

end
imagesc((abs(R)));colormap gray
title('Harris detection for tsukuba image','fontsize',14)
%% setting threshold
[x_val,y_val]=find((abs(R)>10e+6)); % change threshold here
perm=randperm(length(x_val));
sel=perm(1:200);
imshow(I,[]); hold on; plot(y_val,x_val,'ro'); % x and y are different in plot and image
A=corner(I,'Harris');
figure,
imshow(I,[]); hold on; plot(A(:,1),A(:,2),'ro'); % to compare
```

TRANSLATION MATRIX

Here is the function for calculating Translation matrix for simple similarity transformation.

```
function [ t_x,t_y ] = tranlation( matchedPoints1,matchedPoints2 )
diff=matchedPoints1-matchedPoints2;
t_x=mode(round(diff(:,1))); %find the most repeated numbers(round value)
t_y=mode(round(diff(:,2)));


end
```

ROTATION AND SCALING MATRIX

Here is the function for calculating Rotation and scaling matrix for simple similarity transformation.

```
function [ S,res ] = Rotation_S( pointA,pointB )
 syms rotateangle scale

 eqns=[pointB(1)==cos(rotateangle)*scale*pointA(1)+sin(rotateangle)*scale*pointA(2);
     pointB(2)==-sin(rotateangle)*scale*pointA(1)+cos(rotateangle)*scale*pointA(2)];

  S = solve(eqns, [rotateangle,scale]);
  res=[round((S.rotateangle)),round(S.scale)];
  % need to use radtodeg for obtained angel to change it from radians to degree
  % only care about first row in res
end
```

HOMOGENEOUS MATRIX& FUNDAMENTAL MATRIX

Here is the function for calculating homogeneous matrix. Input pointA has coordinates for 4 points in image A, and pointB input has coordinates for 4 corresponding points in image B. Inside the code, 8 equations are used to calculate 8 coefficients in homogeneous matrix. And output h is the vector of all 8 coefficients.

```
function [ S,h ] = homogeneous_equ( pointA,pointB )
syms h11 h22 h13 h21 h12 h23 h31 h32
eqns = [-h11*pointB(1,1)-h13-h12*pointB(1,2)+h31*(pointA(1,1)*pointB(1,1))
    +h32*(pointA(1,1)*pointB(1,2))+pointA(1,1) == 0;
        -h21*pointB(1,1)-h22*pointB(1,2)-h23+h31*(pointA(1,2)*pointB(1,1))
        +h32*(pointA(1,2)*pointB(1,2))+pointA(1,2)==0;
        -h11*pointB(2,1)-h13-h12*pointB(2,2)+h31*(pointA(2,1)*pointB(2,1))
        +h32*(pointA(2,1)*pointB(2,2))+pointA(2,1) == 0;
        -h21*pointB(2,1)-h22*pointB(2,2)-h23+h31*(pointA(2,2)*pointB(2,1))
        +h32*(pointA(2,2)*pointB(2,2))+pointA(2,2)==0;
        -h11*pointB(3,1)-h13-h12*pointB(3,2)+h31*(pointA(3,1)*pointB(3,1))
        +h32*(pointA(3,1)*pointB(3,2))+pointA(3,1) == 0;
        -h21*pointB(3,1)-h22*pointB(3,2)-h23+h31*(pointA(3,2)*pointB(3,1))
        +h32*(pointA(3,2)*pointB(3,2))+pointA(3,2)==0;
        -h11*pointB(4,1)-h13-h12*pointB(4,2)+h31*(pointA(4,1)*pointB(4,1))
        +h32*(pointA(4,1)*pointB(4,2))+pointA(4,1) == 0;
        -h21*pointB(4,1)-h22*pointB(4,2)-h23+h31*(pointA(4,2)*pointB(4,1))
        +h32*(pointA(4,2)*pointB(4,2))+pointA(4,2)==0;];

        S = solve(eqns, [h11 h12 h13 h21 h22 h23 h31 h32 ]);
        h=[round(S.h11),round(S.h12),round(S.h13),round(S.h21),round(S.h22),
            round(S.h23),round(S.h31),round(S.h32) ];

end
```

Here is the function for calculating fundamental matrix.

```
function [A] = fundamental_mat(pointA,pointB)
num_point=size(pointA,1);
A=zeros(num_point,9);
for i=1:num_point
    A(i,1)=pointB(i,1)*pointA(i,1);
    A(i,2)=pointB(i,2)*pointA(i,1);
    A(i,3)=pointA(i,1);
    A(i,4)=pointA(i,2)*pointB(i,1);
    A(i,5)=pointB(i,2)*pointA(i,2);
    A(i,6)=pointA(i,2);
    A(i,7)=pointB(i,1);
    A(i,8)=pointB(i,2);
    A(i,9)=1;
end

end
```

PROJECTING ROTATION MATRIX

```
image1=imread('1.jpg');
% image1=rgb2gray(image1);
image1=double(image1);
%%
image2=zeros(size(image1));
angle=5;
for x=1:size(image1,1)

   for y=1:size(image1,2)

       new_x=cosd(angle)*x+sind(angle)*y;
       new_y=-sind(angle)*x+cosd(angle)*y;
       if (0<(round(new_x)))&&((round(new_x))<size(image1,1))&&
           (0<(round(new_y)))&&((round(new_y))<size(image1,2))
           if image2(round(new_x),round(new_y))==0
              image2(round(new_x),round(new_y))=image1(x,y);
           end
       end
   end


end

figure;
subplot(1,2,1)
imagesc(image1);colormap gray
title('original image','fontsize',14)
subplot(1,2,2)
imagesc(image2);colormap gray
title(sprintf('rotate %d degree',angle),'fontsize',14)
```

HG



Fig. 15. Corners detection on HG image and resize version

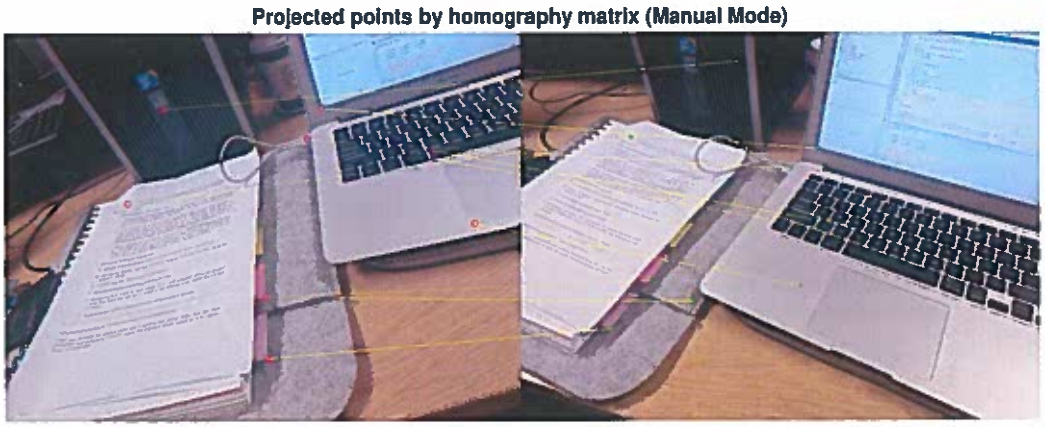


Fig. 16. Projected points by more complex homography

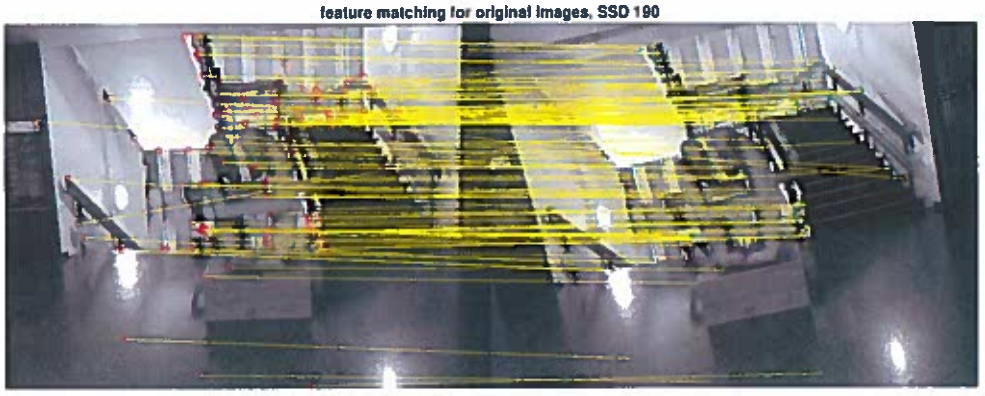


Fig. 17. Matching for original images



Fig. 18. Matching for resized images

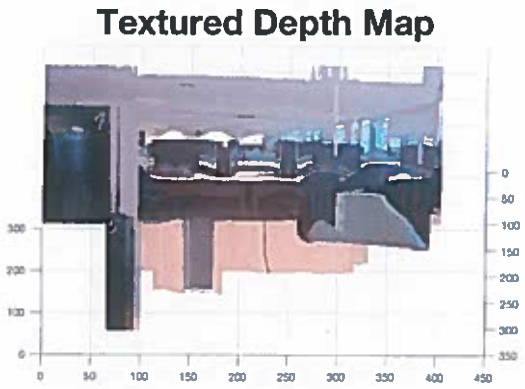


Fig. 19. Textured depth map



Fig. 20. Comparision of original images

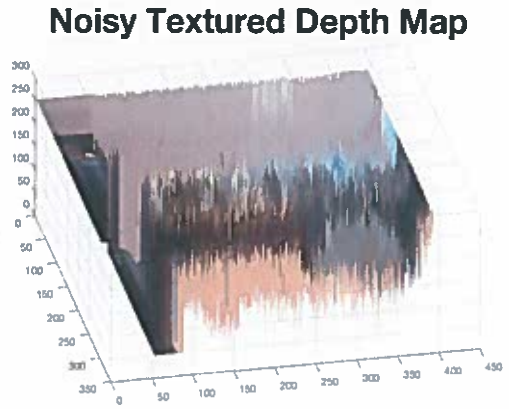


Fig. 21. Noisy Textured depth map

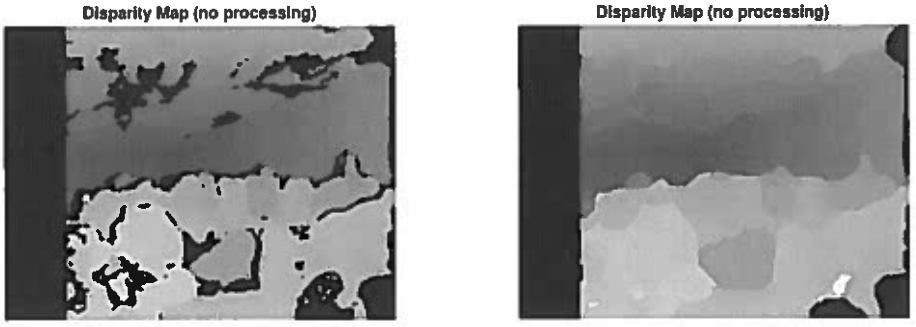


Fig. 22. Disparity Map without processing on the left, and with processing on the right